18. Dokümantasyon Artık Yük Değil: Koddan Canlı Dokümantasyon Üretmek

Eski ve Güncelliğini Yitirmiş Wiki Sayfalarına Son Verme Rehberi
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Geliştiriciler neden dokümantasyon yazmaktan nefret eder ve AI ile dokümantasyonu kodun doğal bir yan ürünü haline nasıl getirebiliriz?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Güncellenmeyen dokümantasyon, hiç yazılmamış dokümantasyondan daha tehlikelidir çünkü yanlış yönlendirir. - 📌 En iyi dokümantasyon kodun hemen yanında yaşayan dokümantasyondur (JSDoc, TypeDoc, OpenAPI/Swagger). - 📌 AI araçları, TypeScript tiplerini ve API endpoint'lerini okuyarak Markdown belgelerini ve API referanslarını anında güncelleyebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Tarihi bir kaleye girdiğinizi ve elinizde 1850 yılından kalma bir harita olduğunu düşünün. Haritada *"Burada gizli bir altın odası var"* yazıyor; gidiyorsunuz ama orada derin bir uçurum var! Harita güncellenmediği için hayatınızı kaybedebilirsiniz.

Yazılım şirketlerindeki eski Confluence sayfaları ve unutulmuş Word dosyaları tam olarak o 1850 yılı haritasıdır. Kod değişir, mimari gelişir ama doküman 2 yıl önceki haliyle kalır.

Modern mühendislikte çözüm **Canlı Dokümantasyon (Living Documentation)** yaklaşımıdır: Dokümantasyon kodun içine yazılır (JSDoc ve Şemalar). Yapay zeka bu şemaları okuyup saniyeler içinde güncel web sitelerine ve markdown sayfalarına dönüştürür.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Koddan Dokümantasyona Otomasyon Piramidi:

```
[Tip ve Fonksiyon Açıklamaları (JSDoc / TypeDoc)]
                    |
                    ▼
[Otomatik Şema Çıkarımı (OpenAPI / Swagger / Zod)]
                    |
                    ▼
[AI İle Zenginleştirme: Kullanım Örnekleri & Senaryolar]
                    |
                    ▼
[Canlı Dokümantasyon Sitesi (Mintlify / Docusaurus / README.md)]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Teknik Yazar ve API Mimarı
Aşağıdaki TypeScript servis fonksiyonunu incele.

## KOD:
```typescript
export async function calculateShippingCost(
  weightKg: number,
  destinationZone: 'LOCAL' | 'NATIONAL' | 'INTERNATIONAL',
  isExpress: boolean = false
): Promise<number> {
  // Mantık...
}
```

GÖREV:

Bu fonksiyon için: 1. Standartlara tam uyumlu TSDoc / JSDoc bloğu oluştur (@param, @returns, @throws). 2. Bir junior geliştiricinin kolayca anlayacağı pratik bir kullanım örneği (code example) ekle. 3. API dokümantasyonu için bir Markdown tablosu ve açıklama paragrafı hazırla.
```

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü Dokümantasyon: Ne parametrelerin anlamı belli ne de olası hatalar
// Kargo hesaplar
export function calc(w: number, z: string, e: boolean) {
  // ...
}
````

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

````
/**
 * Bir siparişin ağırlığına ve teslimat bölgesine göre kargo ücretini hesaplar.
 *
 * @remarks
 * Uluslararası kargolarda ekspres seçeneği ekstra gümrük işlem ücreti içerir.
 *
 * @param weightKg - Paketin kilogram cinsinden ağırlığı (0.01 - 100.0 arası).
 * @param destinationZone - Teslimat bölgesi ('LOCAL', 'NATIONAL', 'INTERNATIONAL').
 * @param isExpress - Hızlı teslimat seçeneği (Varsayılan: false).
 *
 * @returns KDV dahil toplam kargo tutarı (TRY cinsinden).
 *
 * @throws {RangeError} Ağırlık 0 veya negatifse ya da 100 kg'dan büyükse.
 *
 * @example
 * ```typescript
 * const cost = await calculateShippingCost(2.5, 'NATIONAL', true);
 * console.log(`Ödenecek tutar: ${cost} TL`);
 * ```
 */
export async function calculateShippingCost(
  weightKg: number,
  destinationZone: 'LOCAL' | 'NATIONAL' | 'INTERNATIONAL',
  isExpress: boolean = false
): Promise<number> {
  if (weightKg <= 0 || weightKg > 100) {
    throw new RangeError("Paket ağırlığı 0 ile 100 kg arasında olmalıdır.");
  }
  // Hesaplama implementasyonu...
  return 150.0;
}
````

💡 **Kod Analizi:** TSDoc formatında yazılan bu açıklamalar hem IDE içinde fareyle üzerine gelindiğinde zengin ipucu (IntelliSense) olarak görünür, hem de TypeDoc gibi araçlarla tek komutta HTML dokümantasyonuna dönüştürülebilir.
````

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Dokümantasyonu Confluence veya Word dosyalarında tutup koddan koparmak.
  - **Çözüm:** Dokümantasyonu daima kodun içinde (TSDoc) veya aynı repoda markdown dosyalarında tutun.
- **Tuzak:** Kodun ne yaptığını bariz şekilde tekrarlayan anlamsız yorumlar yazmak (Örn: `i++; // i'yi bir artır`).
  - **Çözüm:** Yorumlarda 'ne' yapıldığını değil, 'neden' yapıldığını ve iş kuralı gerekçelerini açıklayın.
- **Tuzak:** API değişikliklerinde dokümantasyonu güncellemeyi unutmak.
  - **Çözüm:** Zod veya OpenAPI şemalarını kullanarak dokümanı koddan otomatik türetin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 18: Canlı Dokümantasyon - Koddan Otomatik Belge Üretmek*

- 🖥️ **Slayt 1:** Neden geleneksel dokümantasyon ölmeye mahkumdur?
- 🖥️ **Slayt 2:** Canlı Dokümantasyon (Living Docs) felsefesi.
- 🖥️ **Slayt 3:** TSDoc, TypeDoc ve Mintlify ile otomatik dokümantasyon üretimi.
- 🖥️ **Slayt 4:** Bölüm 3 Özeti: Takım İşbirliği, Git ve Otomasyonun Gücü.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir TypeScript fonksiyonuna JSDoc ekleyin. TypeDoc veya AI kullanarak bu fonksiyondan anında profesyonel bir `API_REFERENCE.md` sayfası üretin ve önizlemesini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en kritik 2 servis fonksiyonuna eksiksiz TSDoc açıklamaları (@param, @returns, @example) ekleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım geliştirmede 'Canlı Dokümantasyon' (Living Documentation) kavramı neyi ifade eder? - A) Dokümanın canlı yayında okunmasını - B) Dokümantasyonun doğrudan kodun içindeki tiplerden ve açıklamalardan otomatik olarak türetilerek her zaman güncel kalmasını - C) Sadece video şeklinde doküman hazırlanmasını - D) Dokümanın her gün elle yeniden yazılmasını > **Doğru Yanıt:** B > **Açıklama:** Canlı dokümantasyon, kod değiştikçe otomatik güncellenen; tek gerçeklik kaynağının (Single Source of Truth) kod olduğu yaklaşımdır.

Soru 2: TSDoc veya JSDoc bloklarında `@example` etiketinin temel amacı nedir? - A) Fonksiyonu silmek - B) Diğer geliştiricilere fonksiyonun nasıl çağrılacağına dair çalışan, somut bir kod örneği sunmak - C) Sadece CSS renklerini tanımlamak - D) Bilgisayarı hızlandırmak > **Doğru Yanıt:** B > **Açıklama:** `@example` etiketi, geliştiricilerin API'yi nasıl kullanacağını hemen görmesini sağlayan en etkili dokümantasyon aracıdır.